Modified PRISM Architecture & Backtest

Two-stage equity prediction without a reconstruction decoder

Six positive folds. Ranking evidence remains inconclusive.

Aggregate Earnings	Sharpe (as logged)	Rank IC
+20.7503%	+0.09051	-0.000360

The current Kaggle log records completed training and evaluation across six expanding-window folds, covering 837 test dates. It demonstrates that the pipeline ran and reported positive Earnings under this configuration. It does not yet establish stable alpha or superiority over existing models.

An earlier eight-fold experiment reported -5.7801% Earnings. Its training/validation splits and test periods differ. The 26.5304 percentage-point difference cannot be attributed to improved model capability.

Evidence boundary

This report checks the current raw log, download(1).txt, and uses result.txt as the earlier experiment summary. Architecture and formula descriptions refer to the earlier AI-assisted train_prism.py. The exact executed Kaggle source, daily predictions and model weights were not supplied; no independent backtest was rerun.

Research decisions and AI assistance

The researcher proposed the research direction and the decision to remove the reconstruction decoder. ChatGPT assisted implementation, debugging, training integration, and preparation of this report and its chart. Substantial portions of the integrated script are AI-generated. This project is not presented as entirely handwritten code or a complete reproduction of the original paper.

Report date: 2026-11-03 (Asia/Taipei) | English edition | Log-based review

01 Architecture and research modifications

Shapes below describe the reference script, not a line-by-line verification of the final Kaggle executable. N denotes stocks on the same prediction date. The observed input uses 42 time steps and 6 features.

Stage	Data flow and tensor shapes
Stage 1	Input (N,42,6) → RevIN → MLP → GRU temporal summary (N,128) → cross-stock Spatial Transformer → VQ (N,128) → PredictionLinear → (N,)
Stage 2	Freeze the Stage 1 encoder and VQ to obtain codewords (N,128). Project history and codewords to 64 dimensions; prepend the codeword token to form (N,43,64) → Temporal Transformer → first token (N,64).
Fusion and prediction	Apply separate LayerNorms to the first token and original codeword; fuse to (N,64). Codeword-guided routing selects top-2 of 8 experts → combined representation (N,64) → final prediction (N,).

Why remove the reconstruction decoder?

The modified objective no longer requires the representation to reconstruct historical inputs. Prediction and VQ auxiliary objectives remain. The hypothesis is that the codeword can focus more on information useful for future prediction. This benefit has not been demonstrated: a matched-period ablation with and without reconstruction is still required.

Reference implementation details

The codebook contains 512 vectors of dimension 128, selected using cosine similarity, with a straight-through estimator. The temporal encoder has 3 layers and 2 heads; RoPE transforms Q and K within each attention layer. Stage 2 trains the temporal model, MoE and prediction head while keeping the code encoder and codebook frozen.

Stage 1 loss = prediction MSE + 0.25 × commitment loss + codebook loss + contrastive loss. The reference contrastive objective targets the selected code for each stock; it is not claimed to match the paper’s contrastive construction exactly.

Stage 2 loss = prediction MSE + 0.01 × E × sum(Fj × Wj), where E=8, Fj is the fraction of stocks selecting expert j, and Wj is its mean gate weight including zeros. This is not the CV^2 objective; the coefficient is already included in the auxiliary loss.

Spatial attention compares stocks within a date. Temporal attention operates over each stock’s 43 tokens. These attention axes must remain distinct.

02 Data, splits and training evidence

Item	Verified from the current log unless noted
Data	2,332,531 rows; Open, High, Low, Close, Volume, AdjustmentFactor.
Input and portfolio	42 time steps; 200 long stocks at +50% total weight and 200 short stocks at -50%. Gross exposure 100%; net exposure 0%.
Splitting	220 calendar days: user-specified, consistent with adjacent test boundaries. Six expanding-window folds.
GPU	Two Tesla T4 GPUs detected. Model, inputs and gradients are recorded on cuda:0; no evidence of two-GPU parallel execution.
Environment	PyTorch 2.11.0+cu130 and CUDA 13.0, as reported in the log.
Training	Up to 50 epochs per stage; stop after 10 epochs without sufficient validation improvement. Approximately 94 minutes of recorded execution.
Smoke test	Stage 1/2 forward, backward, validation, checkpoint and resume tests passed. The 0.907 GiB peak applies only to the smoke test.

Fold	Validation start	Test boundary	Fit dates	Val dates	Test dates
1	2017-07-05	2017-08-09	82	22	148
2	2017-12-27	2018-03-17	201	51	148
3	2018-06-22	2018-10-23	319	81	143
4	2018-12-07	2019-05-31	434	109	145
5	2019-06-07	2020-01-06	550	138	147
6	2019-11-28	2020-08-13	667	168	106

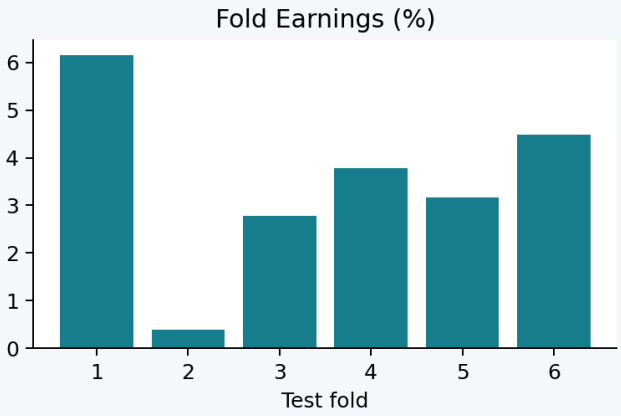
Boundaries are not necessarily trading dates. Minimum stocks per test date range from 1,840 to 1,930 across folds. Fit and validation counts are date counts from the preflight table, not stock-sample counts.

Reference training protocol

The earlier script uses chronological inner validation, boundary-label purging, same-date stock batches and fresh models per fold, selecting the best weights by validation prediction MSE. The log confirms validation and a resume test, but all leakage controls still require verification against the exact executed source.

03 Fold results and historical comparison

Fold	Earnings (%)	Sharpe	Rank IC
1	+6.1584	+0.18769	-0.00554
2	+0.3918	+0.01147	+0.00363
3	+2.7808	+0.12293	+0.00581
4	+3.7818	+0.10682	+0.00918
5	+3.1587	+0.04727	-0.01003
6	+4.4789	+0.18994	-0.00672
Aggregate	+20.7503	+0.09051	-0.000360





Comparison	Earlier: eight-fold summary	Current: six-fold raw log
Earnings	-5.7801%	+20.7503%
Sharpe / Rank IC	-0.02707 / +0.000071	+0.09051 / -0.000360
Positive folds	3 / 8	6 / 6

The sum of unrounded fold Earnings is 20.75033265%, compared with the logged aggregate of 20.75033188%: a difference of approximately 0.00000077 percentage points. These figures are consistent with additive aggregation, allowing for floating-point rounding. Aggregate Sharpe and Rank IC cannot be obtained by simply adding fold values.

The experiments use different test periods, so this is a record of two runs, not a controlled model comparison. Chart bars represent entire test folds, not daily returns or an equity curve. Six positive folds do not imply a 100% daily hit rate.

04 Limitations, diagnostics and reproducibility

Priority 1: constant inputs and ranking ties

Log lines 439-440 and 450-451 contain ConstantInputWarning. Prediction or Target may have zero cross-sectional variation on some dates; the warning alone does not identify which. Record daily prediction standard deviation, unique-value count, Target standard deviation and the number of valid Rank IC dates. Check whether ties select stocks according to security code or row order.

Priority 2: concentration in expert usage

Several Stage 2 validation epochs report val_aux=0.08 while train_aux is around 0.02. If the executed source retains the reference top-2 load formula, 0.08 is consistent with all stocks selecting the same pair of experts. This is a diagnostic clue, not proof of expert collapse. Per-expert selection frequencies and gating-weight distributions are needed.

Metric definitions and tradability

In the reference script, Earnings is the arithmetic sum of daily weighted Targets; Sharpe is the unannualized daily mean divided by sample standard deviation; Rank IC averages daily Spearman correlations while skipping NaNs. These are reference definitions, not verified specifications of the final Kaggle executable. Trading costs, slippage, turnover and holding-period alignment are not established. Earnings should not be described as realized compounded investment returns.

Next experiment: hold comparison conditions fixed

First obtain daily predictions, Targets, dates and security codes, recompute metrics and resolve constant-input dates. Then use common test periods, features and seeds to compare Stage 1 only, no VQ, no MoE and reconstruction-enabled variants. Add zero-prediction, simple linear and random-ranking baselines; evaluate multiple seeds, turnover and costs.

Evidence and reproducibility files

details/kaggle_220day_log.txt preserves the current log; aggregate results appear in lines 452-459. details/previous_180day_summary.txt is the supplied earlier summary, not its raw log. fold_metrics.csv and summary.json contain extracted results. train_prism_reference.py is the earlier AI-assisted source: its default SPLIT_DAYS=180 and it lacks the full resume additions visible in the current log. It must not be treated as the executable that reproduced the current six-fold results.

Dataset CSV SHA-256 reported by the current log:

bf774a86f834e5338bba74f2356ebb750f93a1cc4e89b064ee07251e8ed851db

Not supplied: final Kaggle notebook/script, daily predictions, weights and complete experiment configuration. Maximum drawdown, daily hit rate, equity curves and statistical significance were not computed or fabricated.